Dominio 2: progettazione degli strumenti e integrazione MCP (18%)

Questo dominio tratta la progettazione di interfacce per strumenti che gli LLM possano usare in modo affidabile, la gestione degli errori degli strumenti, la distribuzione degli strumenti tra agenti e la configurazione e il funzionamento dei server MCP.

2.1 Progettazione delle interfacce degli strumenti

Le descrizioni degli strumenti sono fondamentali

Le descrizioni degli strumenti sono il **meccanismo principale** che Claude usa per decidere quale strumento chiamare. La descrizione non è semplice documentazione: è il criterio di selezione.

Descrizione errata (porta a una selezione inaffidabile):

```
{
  "name": "search",
  "description": "Cerca elementi"
}
```

Descrizione corretta:

```
{
  "name": "search_knowledge_base",
  "description": "Cerca nella knowledge base interna articoli di supporto, FAQ e guide alla risoluzione dei problemi. Usalo quando il cliente pone una domanda che potrebbe avere una risposta già documentata. L'input dovrebbe essere una query in linguaggio naturale che descriva il problema del cliente. Restituisce gli articoli corrispondenti ordinati per rilevanza, con titoli, estratti e ID degli articoli."
}
```

Cosa includere nelle descrizioni degli strumenti

1. **Scopo** — Cosa fa lo strumento e quando usarlo
2. **Formato di input** — Come deve essere strutturato l'input, con esempi
3. **Formato di output** — Cosa restituisce lo strumento
4. **Casi limite** — Cosa succede con input non validi o senza risultati
5. **Distinzione** — Quando usare QUESTO strumento invece di strumenti simili

Schema completo della definizione di uno strumento

```
{
  "name": "get_weather",
  "description": "Ottieni il meteo attuale in una determinata località.
Restituisce temperatura, condizioni, umidità e velocità del vento. Usalo quando
l'utente chiede le condizioni meteorologiche attuali. Per le previsioni, usa
invece get_forecast.",
  "input_schema": {
    "type": "object",
    "properties": {
      "location": {
        "type": "string",
        "description": "La città e lo stato o il paese, ad esempio 'San Francisco,
CA' o 'Londra, Regno Unito'"
      },
      "unit": {
        "type": "string",
        "enum": ["celsius", "fahrenheit"],
        "description": "Unità di misura della temperatura. Se non specificata, il
valore predefinito è Celsius."
      }
    },
    "required": ["location"]
  }
}
```

Modalità strict

La modalità strict garantisce che gli input degli strumenti di Claude rispettino esattamente lo schema:

```
{
  "name": "extract_invoice",
  "strict": true,
  "input_schema": {
    "type": "object",
    "properties": {
      "vendor_name": { "type": "string" },
      "invoice_number": { "type": "string" },
      "total_amount": { "type": "number" },
      "currency": { "type": "string", "enum": ["USD", "EUR", "GBP"] }
    },
    "required": ["vendor_name", "invoice_number", "total_amount", "currency"],
    "additionalProperties": false
  }
}
```

Requisiti della modalità strict:

- È necessario impostare `additionalProperties: false`.
- Tutte le proprietà devono comparire in `required`.

- È supportato solo un sottoinsieme di JSON Schema (vedi Dominio 4).

Antipattern: sovrapposizione ambigua tra strumenti

Due strumenti con descrizioni quasi identiche causano errori di instradamento:

```
Strumento 1: "analyze_content" — "Analizza i contenuti per ricavarne informazioni"
Strumento 2: "analyze_document" — "Analizza un documento per ricavarne
informazioni"
```

Claude non riesce a distinguerli in modo affidabile. Soluzione: rinominali per distinguere chiaramente gli scopi oppure uniscili in un unico strumento.

2.2 Risposte di errore strutturate

Struttura degli errori MCP

Quando uno strumento non riesce, la risposta di errore dovrebbe essere strutturata, non una stringa generica:

```
{
  "isError": true,
  "errorCategory": "transient",
  "isRetryable": true,
  "description": "Timeout della connessione al database dopo 30 secondi. La query
riguardava la cronologia degli ordini del cliente n. 12345.",
  "partial_results": null,
  "attempted_action": "SELECT * FROM orders WHERE customer_id = 12345"
}
```

Categorie di errore

Categoria	Ripetibile?	Esempi	Azione dell'agente
Temporaneo	Sì	Timeout, indisponibilità del servizio, limiti di frequenza	Riprova con attesa progressiva
Convalida	No (senza modificare l'input)	Formato email non valido, campo obbligatorio mancante	Correggi l'input e riprova
Business	No	Rimborso oltre il limite previsto dalla policy, account bloccato	Inoltra il caso o informa l'utente
Autorizzazione	No	Accesso insufficiente, token scaduto	Inoltra il caso

Come dovrebbe gestire gli errori l'agente

1. **Errori temporanei:** i subagenti dovrebbero gestire localmente il recupero (riprovando con attesa progressiva). L'errore va propagato solo se il recupero non riesce.

2. **Errori di convalida:** Claude dovrebbe esaminare l'errore, correggere l'input e riprovare.
3. **Errori di business:** Claude dovrebbe spiegare la policy all'utente o inoltrare il caso.
4. **Errori di autorizzazione:** Claude dovrebbe inoltrare il caso a una persona.

Antipattern (risposte errate)

1. **Messaggi di errore generici:** `"Operazione non riuscita"` — nascondono la causa e impediscono un recupero intelligente.
2. **Soppressione silenziosa:** restituire `{ "results": [] }` quando il motore di ricerca è inattivo. Sembra che non ci siano risultati, mentre la ricerca in realtà non è riuscita. L'agente dirà con sicurezza all'utente «Non ho trovato nulla», quando la verità è «Non sono riuscito a eseguire la ricerca».
3. **Messaggi di stato generici:** `"ricerca non disponibile"` senza indicare cosa si è tentato di fare.
4. **Interruzione al primo errore:** terminare un intero workflow articolato in più passaggi perché uno di essi è fallito, invece di usare risultati parziali o tentare alternative.

Distinguere un risultato vuoto da un errore

È un concetto specifico dell'esame:

Situazione	Cosa è successo	Risposta corretta
La ricerca restituisce `[]` con stato 200	Query valida, nessuna corrispondenza	«Nessun risultato per questa query»
La ricerca restituisce un errore di timeout	La query non è stata eseguita	«Al momento non riesco a eseguire la ricerca; riprovo»

Questi casi vanno gestiti in modo diverso. Il primo è un'informazione; il secondo è un errore.

2.3 Distribuzione degli strumenti e parametro tool_choice

Massimo di 4-5 strumenti

Ogni agente dovrebbe avere **al massimo 4-5 strumenti**. Assegnarne 18 a un agente riduce l'affidabilità della selezione: Claude potrebbe scegliere lo strumento sbagliato, chiamarne di non necessari o avere difficoltà a decidere quale usare.

Soluzione per i sistemi complessi: distribuisci gli strumenti tra subagenti specializzati.

```
Invece di:
  Un agente con 18 strumenti

Fai così:
  Coordinatore con: Agent, AskUser
  Subagente di ricerca con: search_web, search_docs, search_db
  Subagente di analisi con: analyze_text, extract_entities, summarize
  Subagente operativo con: send_email, create_ticket, update_record
```

Questa è la risposta prevista dall'esame e il principio di fondo (un agente, un compito mirato) resta valido. È utile conoscere anche i dati più recenti riportati nella documentazione Anthropic.

Oltre la guida d'esame: ricerca degli strumenti

Le indicazioni attualmente pubblicate da Anthropic affermano che **l'accuratezza nella selezione degli strumenti peggiora quando un agente ha a disposizione più o meno 30–50 strumenti** e che una tipica configurazione MCP con più server (GitHub, Slack, Sentry, Grafana, Splunk) consuma circa 55.000 token di contesto nelle definizioni degli strumenti prima ancora di iniziare il lavoro. La soluzione non consiste più soltanto nel «suddividere il lavoro tra subagenti»: si usa lo **strumento di ricerca degli strumenti**, disponibile in versione stabile su Claude API.

```
"tools": [
  { "type": "tool_search_tool_regex_20251119", "name": "tool_search_tool_regex" },
  { "name": "get_weather", "description": "...", "input_schema": { ... },
    "defer_loading": true }
]
```

Funzionamento:

1. Continui a inviare **tutte** le definizioni degli strumenti a ogni richiesta. `defer_loading` controlla cosa entra nella *finestra di contesto*, non cosa trasmetti.
2. Gli strumenti non differiti vengono caricati subito nel contesto; quelli differiti no.
3. Quando Claude ha bisogno di altro, chiama lo strumento di ricerca. L'API restituisce blocchi `tool_reference` e li espande inserendo le definizioni complete direttamente nel messaggio: il prefisso del prompt di sistema non cambia, quindi la **cache del prompt resta valida**.
4. Esistono due varianti: `tool_search_tool_regex_20251119` (Claude scrive pattern per Python `re.search()`, massimo 200 caratteri) e `tool_search_tool_bm25_20251119` (query in linguaggio naturale, massimo 500 caratteri). Entrambe cercano nei nomi, nelle descrizioni, nei nomi degli argomenti e nelle descrizioni degli argomenti.

Regola	Dettaglio
Non differire tutti gli strumenti	Almeno uno deve avere `defer_loading: false`, normalmente lo strumento di ricerca stesso. Altrimenti: **400, "All tools cannot be deferred"**
Mantieni un insieme sempre disponibile	Lascia non differiti i **3–5 strumenti più usati**, così Claude può chiamarli senza un passaggio di ricerca
Limiti	Fino a 10.000 strumenti differiti per richiesta; le ricerche restituiscono 5 corrispondenze per impostazione predefinita (Claude può impostare `limit`, da 1 a 10.000)
Cache	Uno strumento con `defer_loading: true` non può avere anche `cache_control`: errore 400. Imposta il punto di interruzione su uno strumento non differito
Componibilità	Funziona con `strict: true`; la grammatica viene generata a partire dall'intero insieme di strumenti

Regola	Dettaglio
MCP	Non impostare `defer_loading` per ogni singolo strumento: impostalo in `default_config` nella voce `mcp_toolset`

Quando usarlo: almeno 10 strumenti, definizioni degli strumenti superiori a 10.000 token, accuratezza che peggiora all'aumentare degli strumenti o aggregazione di più server MCP. **Quando evitarlo:** meno di 10 strumenti, tutti usati a ogni richiesta oppure definizioni che totalizzano meno di circa 100 token.

Come conciliare le due indicazioni: il limite d'esame di «4-5 strumenti» riguarda quanti strumenti sono *presenti davanti al modello quando deve decidere*. La ricerca degli strumenti non aumenta questo numero: lo mantiene tra 3 e 5, pur consentendo al catalogo sottostante di arrivare a migliaia di strumenti.

Parametro tool_choice

Controlla il modo in cui Claude interagisce con gli strumenti:

Valore	Comportamento	Caso d'uso
`"auto"` (predefinito)	Claude può chiamare uno strumento oppure rispondere con testo	Conversazione generale con uso facoltativo degli strumenti
`"any"`	Claude DEVE chiamare uno strumento, ma può scegliere quale	Garantire un output strutturato
`{"type": "tool", "name": "specific_tool"}`	Claude DEVE chiamare esattamente questo strumento	Forzare una specifica operazione come primo passaggio
`"none"`	Claude non può chiamare alcuno strumento; può solo rispondere con testo	Forzare una risposta testuale mantenendo le definizioni degli strumenti nel contesto

Qualsiasi valore di `tool_choice` può includere anche `"disable_parallel_tool_use": true` per limitare Claude a una sola chiamata a uno strumento per risposta (per impostazione predefinita può emettere più blocchi `tool_use` contemporaneamente).

> **Fable 5.1 / Mythos 5.1 (dal 2026-09-01) e Opus 5.5 (dal 2026-09-22): l'uso forzato degli strumenti non è più supportato.** `{"type": "any"}` e `{"type": "tool", "name": ...}` restituiscono `400 invalid_request_error` (`tool_choice: type "tool" and "any" are not supported for this model`), anche con `count_tokens` e nella Batches API. `auto` e `none` continuano a funzionare; anche `disable_parallel_tool_use` resta compatibile con `auto`. Alternative, in ordine di preferenza:
>
> 1. `tool_choice: {"type": "auto"}` insieme a un'istruzione esplicita che nomina lo strumento (nel turno `user` oppure, se l'applicazione richiede la chiamata, in un messaggio `role: "system"` a metà conversazione); imposta `strict: true` sullo strumento affinché gli argomenti rispettino comunque lo schema.
> 2. Output strutturati (`output_config.format`, Dominio 4 §4.3) se la chiamata forzata serviva solo a ottenere una risposta JSON.

> Sonnet 5, Opus 5 (ora legacy) e la famiglia 4.x mantengono tutti e quattro i valori di `tool_choice`: quindi la risposta d'esame «`any` garantisce una chiamata a uno strumento» è ancora corretta, ma non vale più per nessuno dei due modelli di punta attuali. Prima di darlo per scontato, controlla `client.models.retrieve(id).capabilities`.

Quando usare ciascuna opzione

`"auto"` — Per la maggior parte delle conversazioni. Claude decide se serve uno strumento.

`"any"` — Quando serve garantire un output strutturato. Esempio: sono disponibili più schemi di estrazione (fattura, ricevuta, contratto) e vuoi che Claude scelga quello corretto e restituisca sempre dati strutturati.

Specifica forzata — Quando è necessario eseguire per primo uno strumento specifico. Esempio: chiamare sempre `extract_metadata` prima di qualsiasi passaggio di arricchimento.

`"none"` — Quando un turno deve produrre testo discorsivo (ad esempio, un riepilogo finale) senza rimuovere dalla richiesta le definizioni degli strumenti.

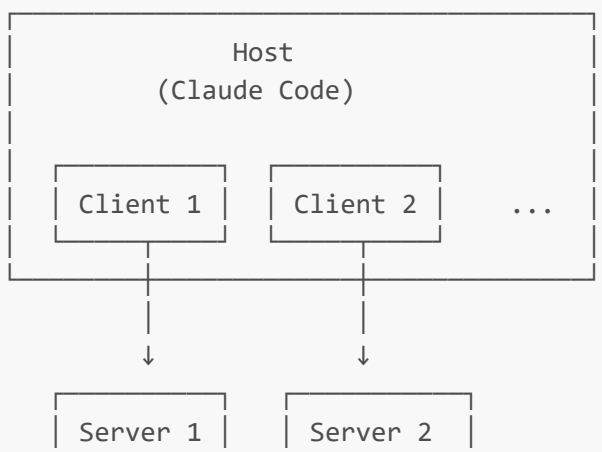
```
# Forza Claude a chiamare uno strumento specifico (Sonnet 5 / Opus 5 / 4.x —
errore 400 con Opus 5.5 e Fable 5.1)
response = client.messages.create(
    model="claude-sonnet-5",
    messages=[...],
    tools=[...],
    tool_choice={"type": "tool", "name": "extract_metadata"}
)
```

2.4 Configurazione dei server MCP

Che cos'è MCP?

Il Model Context Protocol è uno standard aperto per connettere i modelli di IA a origini dati e strumenti esterni. Fornisce a Claude un metodo standardizzato per interagire con database, API, file system e altri servizi.

Architettura



```
    │ (GitHub) │    │ (Postgres)│
    └──────────┘    └───────────┘
```

- **Host**: l'applicazione di IA (Claude Code o un IDE) che coordina i client MCP.
- **Client**: un componente dell'host che mantiene una connessione 1:1 con un server MCP.
- **Server**: un programma che fornisce contesto (strumenti, risorse e prompt) al client.

Tipi di trasporto

Trasporto	`type` nella configurazione	Funzionamento	Quando usarlo
stdio	`"stdio"` (implicito se la voce include `command`)	Input/output standard	Comunicazione con processi locali. Nessun sovraccarico di rete. È il tipo più comune per gli strumenti locali.
HTTP trasmissibile	`"http"` (alias: `"streamable-http"`)	HTTP POST dal client al server; SSE facoltativo per lo streaming	Server remoti e strumenti ospitati nel cloud. È l'unico trasporto remoto che supporta OAuth.
SSE (legacy)	`"sse"`	Endpoint Server-Sent Events	Servizi che espongono ancora solo un endpoint SSE.
WebSocket	`"ws"`	Connessione bidirezionale persistente	Server remoti che inviano eventi senza una richiesta. Autenticazione solo tramite intestazione, senza OAuth.

> **Attenzione alla configurazione:** una voce con `url` ma senza `type` è un errore di configurazione: Claude Code interpreta una voce senza tipo come un server stdio, la ignora e segnala `has a "url" but no "type"`. Imposta sempre `type` per i server remoti.

Dettagli del protocollo MCP

- Usa il formato dei messaggi **JSON-RPC 2.0**.
- Ciclo di vita:
    1. `initialize` — Negoziazione delle capacità (cosa supporta il server)
    2. `notifications/initialized` — Il server conferma di essere pronto
    3. `tools/list` — Il client individua gli strumenti disponibili
    4. `tools/call` — Il client invoca uno strumento

Per impostazione predefinita, Claude Code negozia la revisione del protocollo **2026-07-28** con i server HTTP diretti su tutti i provider (dalla versione v2.1.274; per disattivarla, imposta `MCP_PROTOCOL_NEGOTIATION=legacy`). Questa revisione aggiunge la richiesta di input in modalità URL: un server può chiedere al client di aprire un flusso nel browser invece di mostrare un modulo.

Primitive del server

Primitiva	Descrizione	Esempio
Strumenti	Funzioni eseguibili	`search_database`, `create_issue`
Risorse	Origini dati (cataloghi di contenuti in sola lettura)	Elenchi di file, schemi di database
Prompt	Modelli di interazione	Workflow di prompt predefiniti

Primitive del client

Primitiva	Descrizione
Sampling	Il server richiede al client un completamento LLM
Elicitation	Il server richiede al client un input dell'utente
Roots	Il server chiede al client entro quali URI o limiti del file system può operare

Ambiti di installazione

Esistono tre ambiti. Il file in cui viene registrato un server dipende dall'ambito, non dal file che hai modificato.

Ambito	Dove viene caricato	Condiviso con il team?	Dove viene memorizzato
Locale (predefinito)	Solo il progetto corrente	No	`~/.claude.json`, indicizzato per percorso del progetto
Progetto	Solo il progetto corrente	Sì, tramite controllo versione	`.mcp.json` nella directory principale del progetto
Utente	Tutti i tuoi progetti	No	`~/.claude.json`
Gestito (amministratore)	Ogni sessione dell'organizzazione	Sì, distribuito dal reparto IT	`managedMcpServers` nelle impostazioni gestite (dalla versione v2.1.259): server forniti dall'organizzazione che gli utenti non possono rimuovere

Attenzione alla somiglianza dei nomi: l'ambito *locale* di MCP è registrato in `~/.claude.json` (directory home), mentre le *impostazioni locali* generali si trovano in `.claude/settings.local.json` (directory del progetto). Sono file senza relazione tra loro.

Aggiunta di server dalla CLI:

```
# Server HTTP remoto, condiviso con il team
claude mcp add --transport http --scope project shared-api https://example.com/mcp

# Server remoto con intestazione di autenticazione statica
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer $TOKEN"
```

```
# Server stdio locale: tutto ciò che segue -- viene passato al server senza
modifiche
claude mcp add airtable --env AIRTABLE_API_KEY=YOUR_KEY -- npx -y @example/mcp-
server

# Incolla un blocco di configurazione preparato per un altro client MCP
claude mcp add-json example '{"type":"http","url":"https://mcp.example.com/mcp"}'
```

Configurazione a livello di progetto (`.mcp.json`)

Questo file si trova nella directory principale del progetto ed è soggetto al controllo versione:

```
{
  "mcpServers": {
    "shared-api": {
      "type": "http",
      "url": "https://example.com/mcp",
      "headers": { "Authorization": "Bearer ${API_TOKEN}" },
      "timeout": 600000
    },
    "postgres": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

Dettagli importanti:

- `${DATABASE_URL}` — Espansione di una variabile d'ambiente. Il segreto effettivo NON è contenuto nel file di configurazione. Se la variabile potrebbe non essere impostata, usa `${VAR:-default}`.
- `command` + `args` — Modalità di avvio del processo del server MCP (trasporto stdio).
- `timeout` — Timeout, in millisecondi, per l'esecuzione degli strumenti del singolo server; sostituisce `MCP_TOOL_TIMEOUT` per quel server.
- Nelle sessioni interattive, Claude Code **chiede l'approvazione** prima di usare i server definiti a livello di progetto in `.mcp.json`. `claude -p`, le sessioni Agent SDK e quelle cloud non possono mostrare la richiesta e caricano i server senza chiedere: usa `disabledMcpjsonServers` o `--setting-sources` per escludere un server dalle esecuzioni senza interfaccia; `CLAUDE_CODE_MCP_STARTUP_WAIT_MS` limita il tempo di attesa della prima esecuzione non interattiva per la connessione dei server (`0` = non attendere).
- Alcuni nomi di server sono riservati (`workspace`, `claude-in-chrome`, `computer-use`, `Claude Preview`, `Claude Browser`) e vengono ignorati con un avviso.

Autenticazione per i server remoti

I server HTTP e SSE supportano **OAuth**: esegui `/mcp` durante la sessione (oppure `claude mcp add` e poi autentica) e Claude Code gestirà il flusso nel browser e il rinnovo del token. Per i servizi senza OAuth, passa un token statico tramite `headers` oppure, se la credenziale ha breve durata, generane uno al momento della connessione con `headersHelper` (un comando il cui output standard diventa il valore dell'intestazione). I server WebSocket supportano solo l'autenticazione tramite intestazione.

La ricerca degli strumenti è attiva per impostazione predefinita in Claude Code

Per impostazione predefinita, le definizioni degli strumenti MCP vengono **differite**: all'avvio della sessione vengono caricati solo i nomi degli strumenti e le istruzioni del server; Claude cerca il resto quando serve. Per questo Claude Code non impone un limite di strumenti per server: il limite pratico è il budget di contesto.

`ENABLE_TOOL_SEARCH`	**Comportamento**
(non impostato)	Tutti gli strumenti MCP sono differiti e caricati su richiesta
`true`	Forza il differimento (invia l'intestazione beta anche tramite proxy)
`auto` / `auto:N`	Carica subito le definizioni finché il totale di quelle differite resta sotto il 10% (o N%) della finestra di contesto; oltre tale soglia le differisce
`false`	Carica subito tutti gli strumenti MCP

Imposta `"alwaysLoad": true` nella voce di un server per escluderlo dal differimento se Claude deve usare i suoi strumenti a ogni turno. Chi sviluppa un server dovrebbe scrivere **istruzioni del server** chiare: quando la ricerca degli strumenti è attiva, Claude le usa per decidere se cercare o meno sul server. Claude Code tronca a 2.048 caratteri ciascuna le descrizioni degli strumenti e le istruzioni del server (`CLAUDE_CODE_MAX_MCP_DESCRIPTION_LENGTH` modifica il limite, dalla versione v2.1.280).

Limiti dell'output

Claude Code mostra un avviso quando l'output di uno strumento MCP supera **10.000 token** e, per impostazione predefinita, lo tronca a **25.000 token**. Puoi aumentare il limite con `MAX_MCP_OUTPUT_TOKENS`; la soglia dell'avviso è fissa. Ecco perché è importante ridurre l'output degli strumenti (Dominio 5): un server che produce molto testo può perdere silenziosamente la parte finale della propria risposta.

Risorse MCP

Le risorse espongono cataloghi di contenuti e riducono le chiamate esplorative agli strumenti. Invece di cercare alla cieca, Claude può consultare le risorse disponibili:

```
{
  "resources": [
    {
      "uri": "file:///project/docs/api-reference.md",
      "name": "Riferimento API",
      "mimeType": "text/markdown"
    }
  ]
}
```

2.5 Strumenti integrati

Strumenti integrati di Claude Code e relative modalità d'uso:

Strumento	Scopo	Quando usarlo
Grep	Cerca modelli nei contenuti dei file	Individuare definizioni di funzioni, messaggi di errore, importazioni e stringhe
Glob	Trova file in base a nome o estensione	Individuare file, ad esempio `**/*.test.ts`, `src/**/config.*`
Read	Carica il contenuto completo di un file	Leggere un file specifico dopo averlo individuato con Grep/Glob
Write	Crea nuovi file	Creare file che ancora non esistono
Edit	Applica modifiche mirate	Modificare sezioni specifiche di file esistenti (usa una corrispondenza testuale univoca)
Bash	Esegue comandi nel terminale	git, npm, esecuzione di script e comandi di sistema
WebSearch / WebFetch	Cerca sul Web; recupera e legge un URL	Consultare documentazione aggiornata e verificare il comportamento di un'API
Agent	Avvia un subagente	Delegare attività circoscritte o esplorative (vedi Dominio 1 §1.3)
Skill	Invoca una skill per nome	Eseguire un workflow predefinito
NotebookEdit	Modifica celle di notebook Jupyter	File `.ipynb`
AskUserQuestion	Pone all'utente una domanda che richiede una risposta per proseguire	Decisioni che può prendere solo l'utente
ToolSearch	Carica su richiesta lo schema di uno strumento differito	Controparte lato client della ricerca strumenti dell'API: gli strumenti MCP e quelli integrati usati raramente vengono forniti solo come nomi, finché Claude non li cerca
SendMessage / ListAgents	Invia messaggi a un subagente, collega o altra sessione attivi o completati	Riprendere un subagente mantenendone intatto il contesto (Dominio 3 §3.4); comunicazione tra sessioni
Workflow	Esegue uno script di workflow dinamico che orchestra molti subagenti	Solo se l'utente lo richiede («usa un workflow», `ultracode`); vedi Dominio 1 §1.3

Schema per esplorare una codebase

Procedura corretta per esplorare una codebase sconosciuta:

```
1. Cerca con Grep i punti di ingresso (funzioni main, definizioni delle route,
esportazioni).
2. Leggi i file dei punti di ingresso.
3. Segui le importazioni: cerca con Grep i moduli referenziati.
4. Leggi quei file.
5. Costruisci la comprensione in modo iterativo.
```

Antipattern: leggere subito tutti i file. Spreca contesto e non favorisce la comprensione.

Quando Edit non riesce

Lo strumento Edit funziona cercando stringhe di testo univoche. Se il testo non è univoco nel file, la modifica non riesce.

Alternativa: usa Read per ottenere il contenuto completo del file, poi Write per sostituire l'intero file con la versione modificata.

2.6 Strumenti definiti da Anthropic e strumenti lato server

Non tutti gli strumenti sono scritti da te. Alcuni sono **definiti da Anthropic** (senza schema: dichiari `type` e `name` e Claude conosce l'interfaccia); altri sono **lato server** (vengono eseguiti sull'infrastruttura Anthropic e restituiscono i risultati come blocchi di contenuto nella stessa risposta, senza un loop di esecuzione lato client).

Strumento	`type`	**Lato client o server**	**Blocco del risultato**
Ricerca Web	`web_search_20260209`	Server	`web_search_tool_result`
Recupero di contenuti Web	`web_fetch_20260209`	Server	`web_fetch_tool_result`
Esecuzione del codice	`code_execution_20260521`	Server	`bash_code_execution_tool_result` (`.content.stdout`)
Ricerca degli strumenti (regex / BM25)	`tool_search_tool_regex_20251119` / `..._bm25_20251119`	Server	`tool_search_tool_result`

Strumento	`type`	Lato client o server	Blocco del risultato
Memoria	`memory_20250818`	Client (implementi tu le operazioni sui file)	`tool_result` standard
Bash	`bash_20250124`	Client	`tool_result` standard
Editor di testo	`text_editor_20250728`	Client	`tool_result` standard

Dettagli da ricordare per l'esame o per una revisione progettuale:

- **Includi la versione nella stringa `type`.** `web_search_20260209` / `web_fetch_20260209` aggiungono filtri dinamici e richiedono Opus 4.6+ / Sonnet 4.6+; i modelli precedenti usano le versioni base `web_search_20250305` / `web_fetch_20250910`. Non dichiarare anche `code_execution` insieme alle varianti `_20260209`: eseguono già codice internamente e un secondo ambiente di esecuzione confonde il modello.
- **Web fetch recupera solo URL già presenti nella conversazione.** Non è un crawler.
- **Gli errori degli strumenti lato server non generano eccezioni.** Restituiscono HTTP 200 con un oggetto di errore nel blocco del risultato (ad esempio `{"error_code": "max_uses_exceeded"}`). Per la ricerca Web, `content` è una *lista* in caso di successo e un *oggetto* in caso di errore: controllalo prima di indicizzarlo. È la stessa distinzione tra «vuoto» e «non riuscito» descritta in §2.2, a livello API.
- `pause_turn` segnala che il loop di uno strumento lato server ha raggiunto il limite interno. Invia di nuovo la risposta per continuare (Dominio 1 §1.1).
- **Bash ed editor di testo non hanno uno schema predefinito.** Dichiarare un proprio strumento personalizzato chiamato `"bash"` con `input_schema` crea uno strumento *diverso*.
- **Strumento Advisor:** il suo `model` deve essere almeno altrettanto capace del `model` di primo livello della richiesta (ad esempio, esecutore `claude-sonnet-5` → advisor `claude-opus-5`). Una combinazione non valida restituisce 400.

Uso parallelo degli strumenti

Per impostazione predefinita, Claude può emettere più blocchi `tool_use` in un singolo messaggio dell'assistente. Eseguili in parallelo e restituisci **tutti** i blocchi `tool_result` in un **unico** messaggio utente. Suddividerli in più messaggi insegna silenziosamente a Claude a non effettuare più chiamate parallele. Se uno strumento non è riuscito, restituisci il suo `tool_result` con `is_error: true`: non ometterlo mai.

Chiamate programmatiche agli strumenti

Claude può chiamare uno strumento personalizzato *tuo* dall'interno dell'ambiente isolato per l'esecuzione del codice, evitando di passare avanti e indietro dalla conversazione. È utile quando uno strumento restituisce risultati voluminosi che altrimenti saturerebbero il contesto. Dichiara `{"type": "code_execution_20260120", "name": "code_execution"}` e imposta `"allowed_callers": ["code_execution_20260120"]` sullo strumento personalizzato. Per rispondere a una chiamata

programmatica in sospeso, il messaggio utente deve contenere **solo** blocchi `tool_result`, senza testo. Non è compatibile con `strict: true`, `disable_parallel_tool_use`, `tool_choice` forzato o strumenti MCP.

MCP dal lato API: il connettore MCP

La sezione MCP del Dominio 2 descrive il punto di vista di Claude Code (un host che gestisce i propri client). Anche Messages API può connettersi direttamente a server MCP remoti. Servono **entrambe le parti**: passare solo `mcp_servers` causa un errore di convalida.

```
client.beta.messages.create(
    model="claude-opus-5-5",
    betas=["mcp-client-2025-11-20"],
    mcp_servers=[{"type": "url", "url": "https://mcp.example.com/mcp", "name":
"example"}],
    tools=[{"type": "mcp_toolset", "mcp_server_name": "example"}],
    messages=[...],
)
```

Imposta `defer_loading` una volta in `default_config` nella voce `mcp_toolset` (oppure per singolo strumento in `configs`), anziché nelle definizioni dei singoli strumenti.

Modificare gli strumenti durante la conversazione senza invalidare la cache

L'array `tools` si trova *prima* di `system` nel prefisso memorizzato nella cache; modificarlo tra un turno e l'altro invalida quindi la cache dell'intera conversazione (Dominio 4 §4.8). Nei modelli che supportano messaggi `system` a metà conversazione (Fable 5 / 5.1, Mythos 5 / 5.1, Opus 5.5, Opus 5, Opus 4.8; Sonnet 5 è escluso), dichiara subito l'intero insieme di strumenti e poi *aggiungi o ritira* strumenti tramite blocchi `tool_addition` / `tool_removal` in un messaggio `role: "system"` (intestazione beta `mid-conversation-tool-changes-2026-07-01`; il campo `tool` di ogni blocco è un `tool_reference` per nome oppure, per gli strumenti del connettore, `mcp_tool_reference` / `mcp_toolset_reference`; indicare uno strumento assente da `tools` causa un errore 400, `tool_reference_unresolved`). Uno strumento dichiarato con `defer_loading: true` resta nascosto finché un blocco `tool_addition` non lo rende disponibile. Con Fable 5.1 e Opus 5.5 è anche l'unico modo di cambiare strumenti senza compromettere la cronologia, perché ricreare `tools` equivale a modificare turni precedenti e invalida i successivi blocchi di ragionamento.

Definire strumenti in un messaggio (beta `inline-tools-2026-09-15`, dal 2026-09-22)

La forma con riferimenti presuppone che tutti gli strumenti fossero noti già alla prima richiesta. La forma inline consente invece a un blocco `tool_addition` di contenere la **definizione completa**:

```
{
  "role": "system",
  "content": [{
    "type": "tool_addition",
    "tool": {
      "type": "tool_definition",
      "definition": {
        "name": "db_query",
```

```
        "description": "Esegui una query SQL in sola lettura sul database di
analisi.",
        "input_schema": {"type": "object", "properties": {"sql": {"type":
"string"}}, "required": ["sql"]}
      }
    }
  }]
}
```

Se invii una definizione diversa con lo **stesso nome**, questa sostituisce la precedente da quel momento in poi: così puoi cambiare uno schema o passare a una versione più recente di uno strumento server senza modificare `tools` né la cache del prompt (riutilizzare il nome di uno strumento di *tipo* diverso causa un errore 400, `tool_name_conflict`). Includendo anche l'intestazione `mcp-client-2026-09-15`, la definizione può essere un `mcp_toolset`; la risposta inizia allora con un blocco `mcp_tool_listing` per ogni server interrogato. Restituisci la risposta per fissare quell'elenco di strumenti e non leggere alla cieca `content[0]`. La funzionalità è disponibile in tutti i modelli che supportano le modifiche agli strumenti a metà conversazione, incluso Opus 5.5.

Regole pratiche: dichiara subito ciò che conosci (in `tools`, differito se il modello non deve ancora vederlo) e definisci inline solo ciò che non è noto alla prima richiesta o cambia in seguito; mantieni in `tools` almeno uno strumento non differito, altrimenti la prima definizione inline modifica l'inizio del prompt generato e provoca una perdita completa della cache; `cache_control` va nel blocco oppure nella definizione, non in entrambi, e una definizione differita non può includerlo. Limiti: 10.000 strumenti differiti o definiti nei messaggi e 4 MB di definizioni inviate dopo il primo messaggio dell'utente (`available_tools_limit_exceeded`). Dopo una compattazione su richiesta, le modifiche agli strumenti nell'intervallo riassunto vengono mantenute solo se la richiesta di compattazione includeva anche questa intestazione; il blocco registra allora l'effetto complessivo nel campo `tool_changes` (Dominio 5 §5.7).